

Introduction to numerical methods (2025)

1

Requirements

- No homeworks
- Practice (starting from week 2) small test from lecture / notebook : 50%
- End of semester test : 40% (60 points)
- Practice solution 70% (in pairs)
- Project (presented online) in pairs (100 points)
 - 5: 132 -
 - 4: 112 - 135
 - 3: 104 - 111
 - 2: 64 - 103
- Exemption

Variables

python: dynamic variables, but some
can be defined
e.g. $d = \text{dict}()$

Basic types: int, float, string, file (io...)

- some packages have own data type which can be useful and annoying
e.g. np.int8 $\in [0, 255]$ saves memory
but json package cannot serialize it.

Floating point number

$$0.1 + 0.2 - 0.3 \approx 5.55 \cdot 10^{-12}$$

↳ never test the equality of floating point numbers!!!

Sign	exponent (11)	fraction (52)
s		$f = f_1 \frac{1}{2} + f_2 \frac{1}{2^2} + \dots + f_{52} \frac{1}{2^{52}}$
	$e \in [0-2048]$	

number = $(-1)^s (1 + f) \cdot 2^{e-1023}$

$\frac{1}{2^{52}} \approx 2.2 \cdot 10^{-16}$

Example: $s=1$ $e = 1 \cdot 2^{10} + 1 \cdot 2^1 = 1026$ $f = 1 \cdot \frac{1}{2^1} + 0 = \frac{1}{2}$

$$u = (-1)^1 \cdot \left(1 + \frac{1}{2}\right) \cdot 2^{1026-1023} = (-1) \cdot 1.5 \cdot 2^3 = \underline{\underline{-12}}$$

Array types

- list: indexed elements
 - + mixed type
 - + variable length
 - no bulk operations
 - wasteful memory usage
- dictionary: elements with keys
 - + mixed type
 - + variable length
 - + keys
 - + data transfer between systems (json)
 - inefficient memory usage
 - bulk operations
- numpy array: fixed shape, fixed type
 - + efficient memory usage
 - + bulk operations few down times faster than for loop
 - + reshape, slice
 - add / remove data is slow
 - ↳ reserve new space, copy, add data, free old space

Python is an interpreted language,
avoid for loops whenever it is possible

Example: $N = 10^6$ $p = 10^{-5}$

Change numbers in array a with prob. p . In average only 10 operations but 10^6 random numbers

(1) for i in range(N):

if $\text{up.random.random()} < p$:

$a[i] = \dots$

$\rightarrow \sim \underline{170 \text{ ms}}$ (N random numbers + for loop)

(2) $r = \text{up.random.random}(N) < p$

$a[r] = \dots$

$\rightarrow \sim \underline{6 \text{ ms}}$ (N random numbers)

(3) How many positions need change?

Binomial distribution:

$n = \text{up.random.binomial}(N, p)$

$a[\text{up.random.choice}(N, n)] = \dots$

$n + 1 \times 11$ random numbers

$\rightarrow \underline{0.27 \text{ ms}}$

Linear algebra

Permutations: each of possible ways in which a set can be ordered.

e.g.

123, 132, 213, 231, 312, 321

recursive algorithm

- pick each element successively, place it to the first position then place all the permutation of the rest after it.

Levi-Civita symbol:

$$\epsilon_{a_1, a_2, \dots, a_n} = \begin{cases} +1 & \text{if } (a_1, a_2, \dots, a_n) \text{ is an even perm. of } [1, 2, \dots, n] \\ -1 & \text{if } (a_1, a_2, \dots, a_n) \text{ is an odd perm. of } [1, 2, \dots, n] \\ 0 & \text{otherwise (there are repeating elements)} \end{cases}$$

Matrix multiplication

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & & & a_{mn} \end{pmatrix}$$

$$(A \cdot v)_i = \sum_{j=1}^n A_{ij} v_j$$

$$(AB)_{ij} = C_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad \left. \begin{array}{l} A: n \times n \\ B: p \times n \end{array} \right\} C: p \times n$$

3 for loops

Matrix operations

- never write it in python
- built in ones are fast
- array vs. matrix : functions

↳ sparse matrix

- few non zero elements
- different memory methods

Sparse memory methods (examples)

- sparse coordinate (construction)
- compressed row / column, fast $+$, $*$
fast row / column slicing
- list of list : efficient construction and storage

- Efficiency: numpy & scipy

- built in functions are generally efficient, but single threaded
- they can be made parallel by setting os environment correctly (this could be the default)
- works with any dimensional tensors but optimization is for 2D
e.g. 2d array 4d OP 2d array
faster!